RNNs and LSTMs: Introduction

2026-04-17

Introduction

Recurrent Neural Networks (RNNs) process sequences by maintaining a hidden state that updates at every time step. Long Short-Term Memory units (LSTM; Hochreiter & Schmidhuber, 1997) add gating mechanisms to combat vanishing gradients, enabling long-range dependencies. GRUs (Cho, 2014) are a simpler gated variant.

Prerequisites

Feedforward networks; gradient-based training; sequence tasks.

Theory

Vanilla RNN: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. Gradients through time vanish or explode, limiting long-range dependence.

LSTM adds input, forget, and output gates, plus a memory cell:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t).$$

Gates control what is written, read, and forgotten in the memory cell.

Bidirectional RNNs concatenate forward and backward hidden states – useful for sequence classification.

Assumptions

Inputs are sequential and order-dependent; padding/masking handles variable-length sequences correctly; sufficient context in the hidden state.

R Implementation

```
library(torch)

torch_manual_seed(2026)

# Batch of 4 sequences, each length 10 and feature dim 5
x <- torch_randn(4, 10, 5)

lstm <- nn_lstm(input_size = 5, hidden_size = 8, num_layers = 2,
                batch_first = TRUE)
out  <- lstm(x)             # list: output tensor, (h_n, c_n)

out[[1]]$shape              # [4, 10, 8]
out[[2]][[1]]$shape         # h_n: [2, 4, 8]

# Simple forecasting model: LSTM + linear head
forecaster <- nn_module(
```

```
  initialize = function() {
    self$lstm <- nn_lstm(1, 16, batch_first = TRUE)
    self$fc   <- nn_linear(16, 1)
  },
  forward = function(x) {
    out <- self$lstm(x)[[1]]
    self$fc(out[, -1, ])       # predict from last time step
  }
)

model <- forecaster()
y_seq <- torch_randn(4, 10, 1)
model(y_seq)$shape          # [4, 1]
```

Output & Results

The LSTM returns per-timestep hidden states (shape [batch, seq, hidden]) plus final hidden and cell states.

Interpretation

"A 2-layer LSTM with 8 hidden units processed 10-step sequences; the final hidden state feeds a linear classifier. Transformers now exceed LSTMs on most NLP tasks, but LSTMs remain strong for short-window forecasting and low-data regimes."

Practical Tips

- Use LSTMs or GRUs over vanilla RNNs; the latter are hard to train for any real-world sequence length.
- Pack sequences (`nn_utils_rnn_pack_padded_sequence`) to skip padding computations.
- Gradient clipping (`nn_utils_clip_grad_norm_`) is almost always needed for RNN training stability.
- For short tasks, a 1-D CNN over the sequence often matches or beats an LSTM with faster training.
- For long sequences with global dependencies, transformers beat LSTMs; but they need more data.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis